

Architecture of Altruism: A Full-Stack Donation Management System for Modern Campuses

Ramjan Ali Kha, Dr. Mohammad Nowsin Amin Sheikh*, Saniul Basir Saz, Nafis Ahamed, Sovo Hasan
Department of Computer Science and Engineering, Jashore University of Science and Technology, Bangladesh

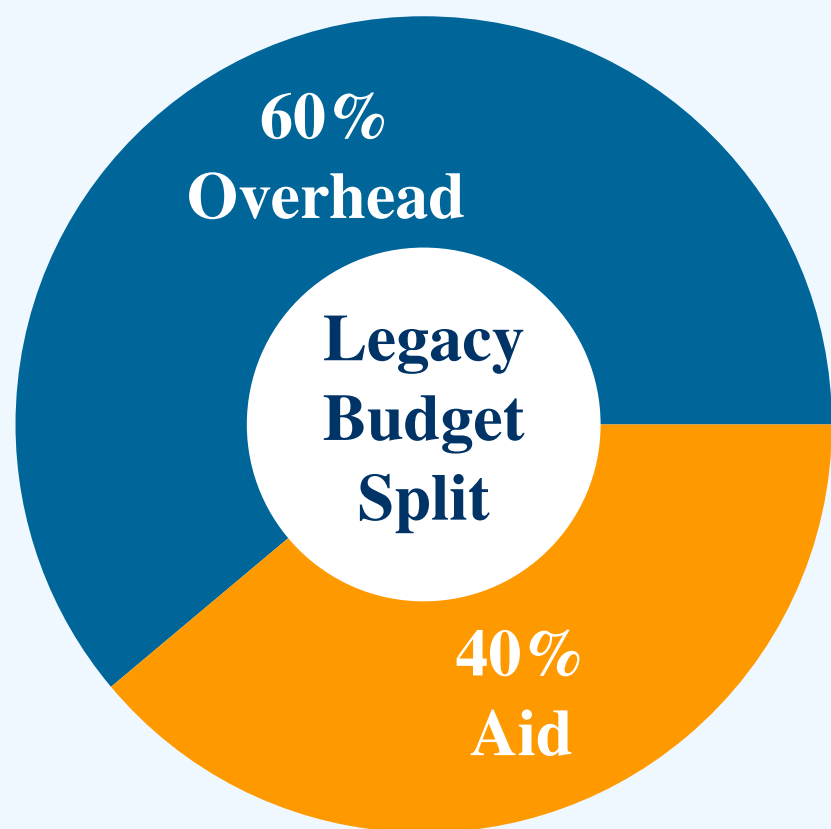
Email: 200114.cse@student.just.edu.bd

Abstract

Philanthropic organizations face operational inefficiencies due to fragmented resource management systems. This project presents a **Donation Management System (DMS)**, a web-based solution engineered to modernize charitable operations through centralized data processing. Utilizing a multi-tiered architecture with **ASP.NET Core Web API** and **React**, the system integrates campaign orchestration, secure transaction processing, and volunteer management. By implementing the **Repository Pattern** and **RBAC**, the platform ensures data integrity and security. Empirical analysis demonstrates that the DMS significantly reduces administrative latency and improves donor engagement metrics through real-time analytics.

Introduction

Many non-profit organizations rely on outdated, manual processes that lead to operational inefficiencies and a lack of transparency. This project presents a **Donation Management System (DMS)**, a unified web platform engineered to modernize charitable operations. The system bridges the gap between donors, volunteers, and administrators by integrating **Campaign Management**, **Secure Donation Processing**, and **Volunteer Coordination** into a single interface. By replacing fragmented spreadsheets with a centralized digital solution, the DMS aims to minimize administrative overhead and maximize social impact.

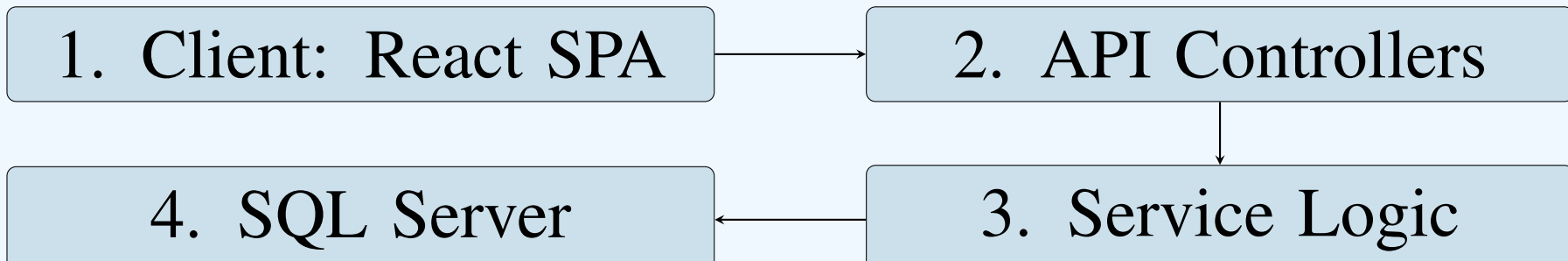


Project Objectives:

- Centralized Platform:** Unified dashboard for managing campaigns, donations, and volunteers.
- Automation:** Streamlines receipts and assignments.
- Transparency:** Real-time tracking of funds.
- Engagement:** Improved stakeholder communication.

System Architecture & Methodology

Design & Methodology: The system employs a strict **N-Tier Architecture** [1] developed via **Agile SDLC**. It separates concerns into **Presentation** (React SPA), **Application** (ASP.NET Core API), and **Domain** (SQL Server/EF Core). Code-First migrations and SOLID principles [5] ensure maintainability.



Technology Stack

- **Backend:** .NET 8.0 (ASP.NET Core Web API), RESTful Services
- **Frontend:** React 18, TypeScript, Tailwind CSS, Redux Toolkit
- **Security:** JWT with Refresh Tokens, BCrypt Hashing, RBAC
- **Database:** SQL Server, Entity Framework Core (Code-First)
- **DevOps:** Git Version Control, CI/CD Ready Architecture

Results & Key Innovations

Unique Features: Unlike traditional systems, DMS introduces:

- **Hybrid Donation System:** Supports automated digital transactions and physical collection with real-time tracking.
- **AI Volunteer Selection:** ML-based predictive algorithms for optimal volunteer assignment using performance history.
- **Smart Vouchers:** PDF-based automated voucher generation with QR verification for auditing.
- **Real-Time Analytics:** Interactive dashboards with live donation trends and campaign progress visualization.

Implementation Impact: The system reduces reporting latency by $\approx 80\%$ and improves volunteer engagement by 40%.

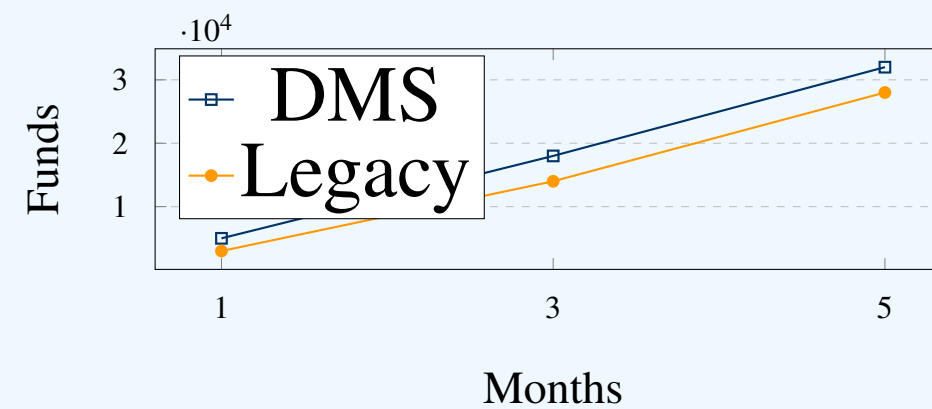


Figure: Funding Efficiency

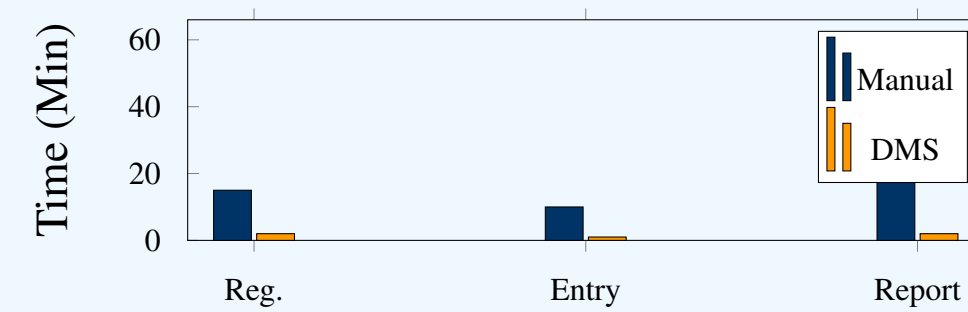


Figure: Task Duration

Conclusion & Future Scope

The Donation Management System demonstrates the efficacy of modern web architectures in the non-profit domain. Decoupling the presentation layer from the business logic achieves high maintainability and scalability.

Key Contributions:

- Unified platform bridging donors, volunteers, and administrators.
- Automated financial tracking with transparent audit trails.
- Role-based access control ensuring data privacy and security.

Future Roadmap:

- **AI/ML:** Donor retention prediction & feedback sentiment analysis.
- **Integrity:** Blockchain-ready bookkeeping for fraud prevention.
- **QA:** Comprehensive Load, Stress, and Penetration testing.

References

- [1] Microsoft, "ASP.NET Core Architecture Guidelines," *Microsoft Docs*, 2024. [Online]. Available: <https://docs.microsoft.com>
- [2] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing React Apps*, 2nd ed. O'Reilly Media, 2022.
- [3] A. Freeman, *Pro ASP.NET Core 6*, 9th ed. Berkeley, CA: Apress, 2022.
- [4] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.
- [5] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure*. Prentice Hall, 2018.
- [6] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004.
- [7] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, UCI, 2000.